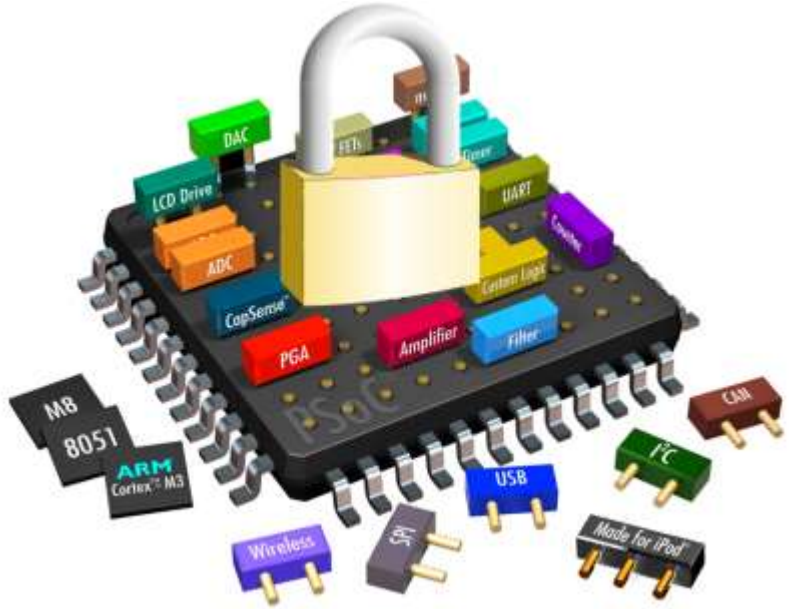
Hardware Security

Intel SGX: Architecture and Security Vulnerabilities



SGX Security Objectives

01

Integrity Protection

SGX must detect any integrity violations of enclave code or data, preventing access to modified content and ensuring execution authenticity.

02

Confidentiality Guarantee

Security-critical code and data within enclaves must remain confidential, protected from unauthorized observation or extraction.

03

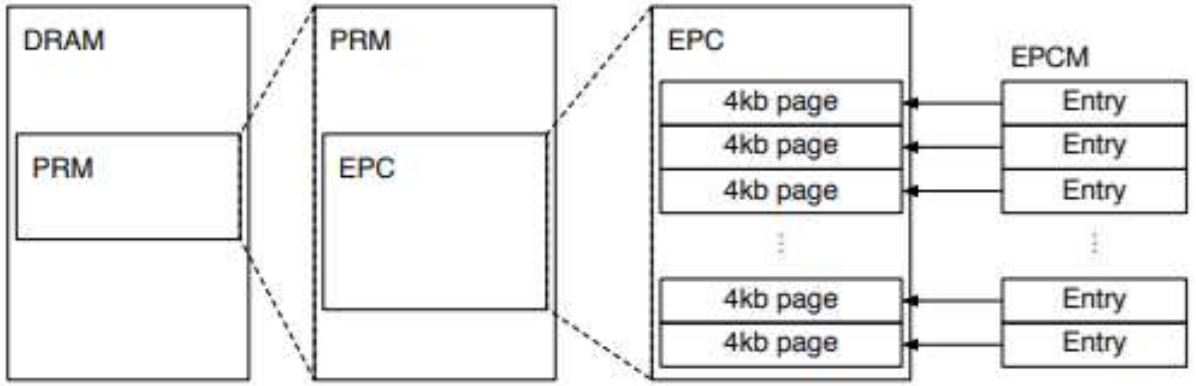
Isolation Assurance

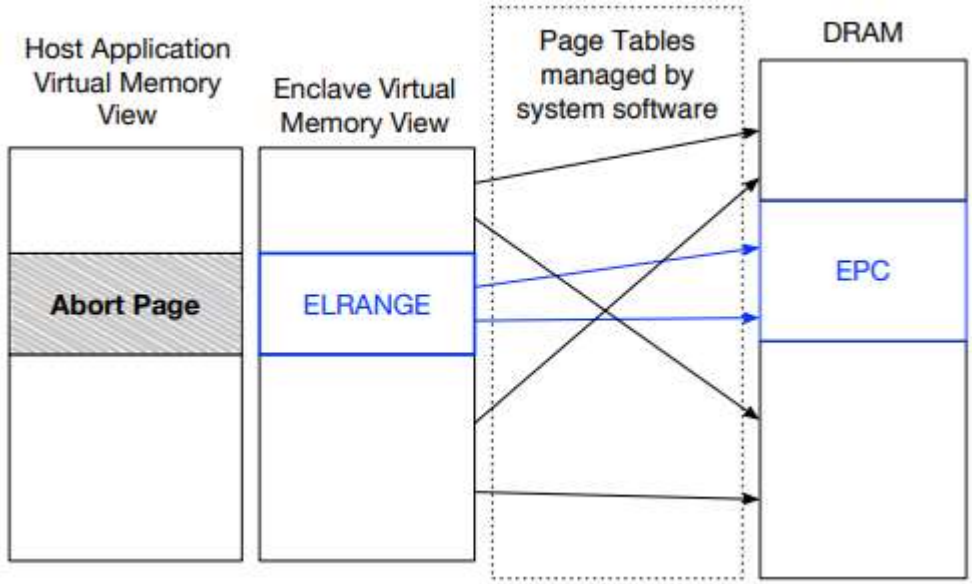
Multiple enclave instances must coexist securely on the same platform with guaranteed isolation between different enclaves.

SGX Execution Model

- SGX applications execute in two distinct regions: a normal untrusted region and a security-critical enclave region.
- The host process initiates an enclave, loads sensitive code and data into protected memory, and calls enclave functions when security-critical operations are needed.
- Only code executing within the enclave can access Processor Reserved Memory (PRM).
- After enclave functions complete, results return to the host process while enclave contents remain protected in isolated memory space.

Memory Organization in SGX





01

Processor Reserved Memory (PRM)

A protected memory region that the CPU shields from all non-enclave accesses, including kernel, hypervisor, SMM, and DMA transfers from peripherals.

02

Enclave Page Cache (EPC)

4KB pages within the PRM that store enclave code and data. System software manages EPC allocation while the CPU tracks each page's state.

03

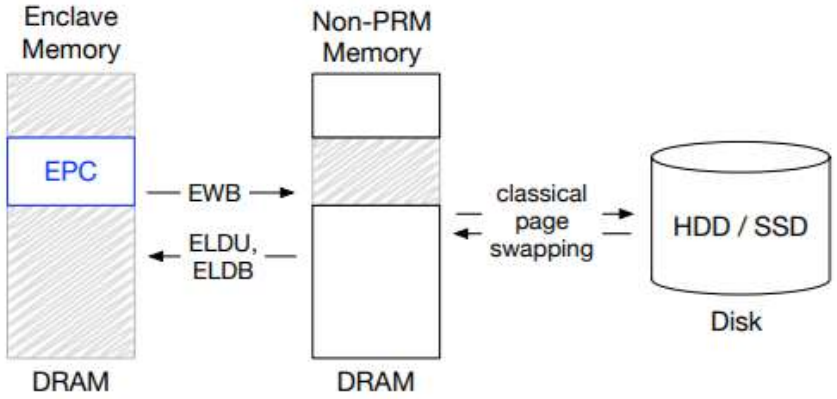
Enclave Page Cache Map (EPCM)

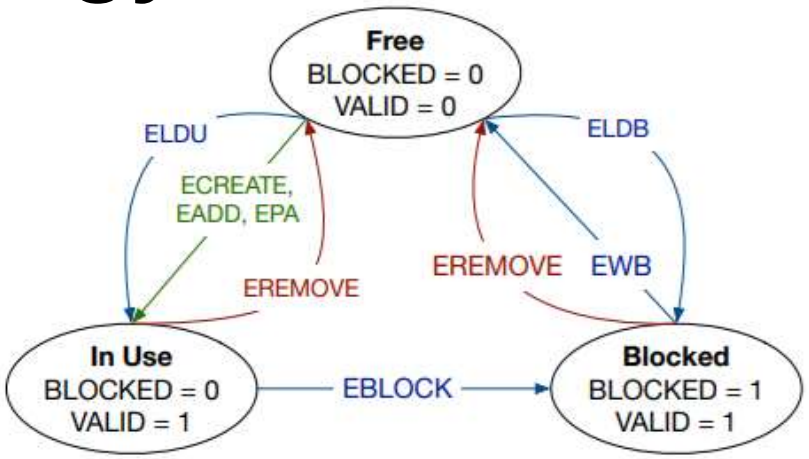
The Enclave Page Cache Map ensures each EPC page belongs to exactly one enclave, preventing unauthorized access and maintaining isolation.

Critical Data Structures

- SECS
  - SGX Enclave Control Structure stores enclave metadata in dedicated EPC pages, accessible only by CPU hardware for enclave management operations.
- TCS
  - Thread Control Structure enables concurrent execution within enclaves, with each TCS stored in a dedicated EPC page for thread management.
- SSA
  - State Save Area securely stores enclave execution context during hardware exceptions, protecting sensitive state from untrusted system software.

EPC Page Eviction Strategy





EBLOCK

Marks pages for eviction and prevents new TLB entries from being created for blocked pages.

ETRACK

Tracks which logical processors need TLB flushes before eviction can safely proceed.

EWB

Encrypts and MACs the page contents, generates a nonce, and writes everything to untrusted DRAM.

ELDU/ELDB

Loads evicted pages back into EPC, verifying integrity and freshness before restoring.

Address Translation Security

Virtual Memory Isolation

Each enclave designates an ELRANGE in its virtual address space for EPC pages. The CPU enforces that enclave code can only access memory within this range or explicitly shared non-EPC memory.

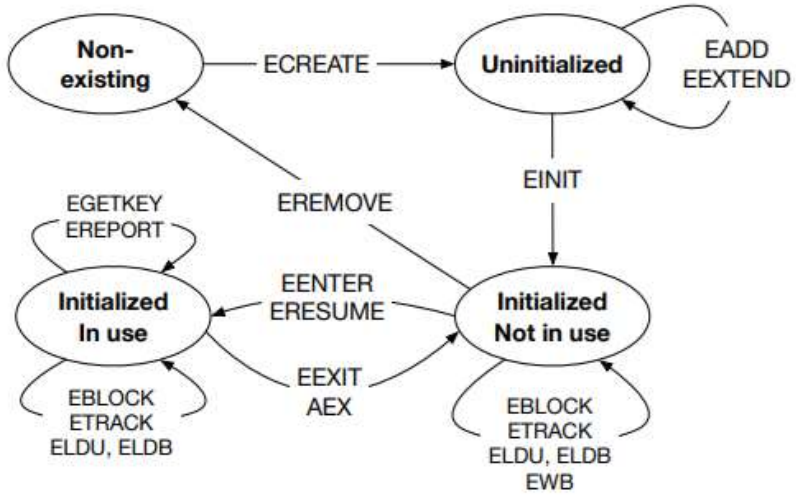
Page Table Verification

While untrusted system software manages page tables, SGX verifies every address translation to ensure it matches the enclave author's intended memory layout and access permissions.

Attack Prevention

SGX's checks prevent malicious OS from performing memory mapping attacks, such as remapping enclave pages to different virtual addresses or modifying access permissions.

Enclave Lifecycle



Creation

ECREATE instruction loads a new SECS into a free EPC page, establishing the enclave's foundation.

Initialization

EINIT instruction finalizes the enclave, enabling execution and preventing further loading operations.

1 2 3 4

Loading

EADD and EEXTEND instructions load initial code and data while updating enclave measurements.

Teardown

EREMOVE instruction frees EPC pages, completing the enclave lifecycle and releasing resources.

Entry and Exit Mechanisms

1

Synchronous Entry

EENTER instruction performs controlled jump to predefined addresses, preventing security bypass attacks.

2

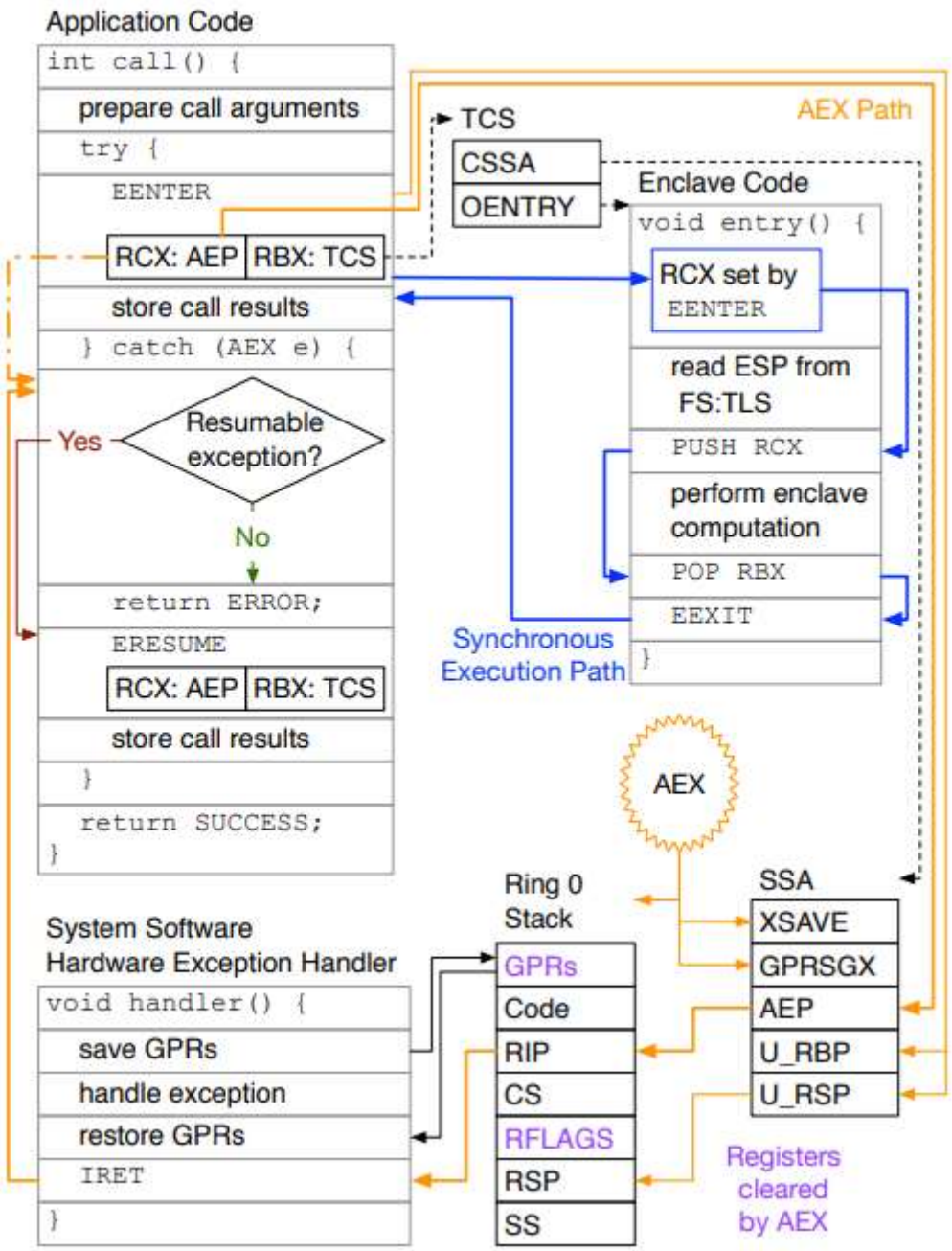
Asynchronous Exit

Hardware exceptions trigger AEX instruction, saving context before invoking exception handler.

3

Resumption

ERESUME instruction restores enclave execution after interrupt handling completes.



Local Attestation

Establish Channel

Prover Enclave obtains Verifier Enclave's measurement through a secure communication channel.

Generate Report

Prover Enclave invokes EREPORT to create a REPORT structure and transmits it to the Verifier.

Verify Report

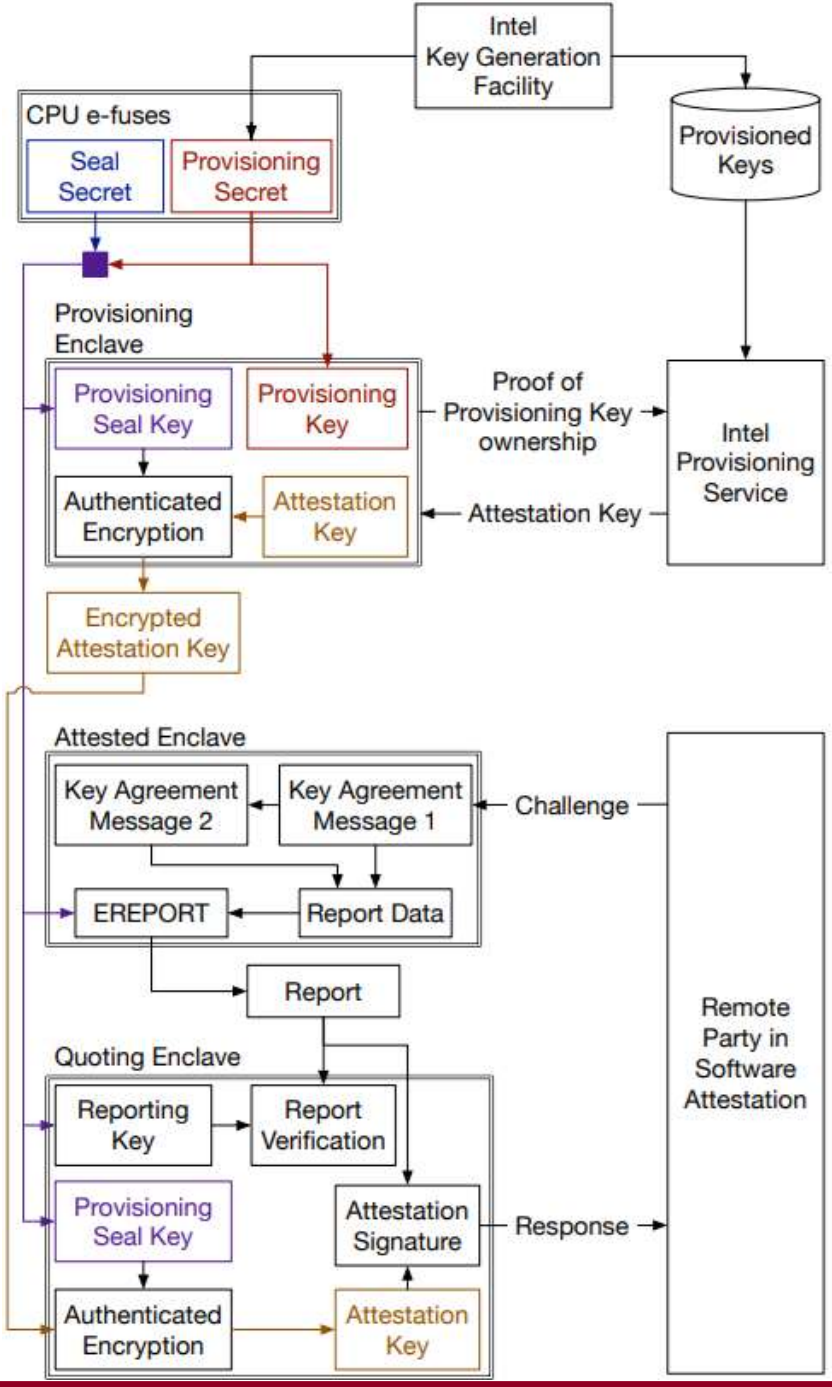
Verifier Enclave retrieves the Report Key using EGETKEY and computes MAC to verify the REPORT's authenticity.

Confirm Platform

Valid MAC confirms both enclaves execute on the same platform, establishing local trust.

Remote Attestation

- Remote attestation extends local attestation to prove enclave identity to remote parties.
- The process utilizes a special Quoting Enclave (QE) to verify and transform REPORT structures into signed QUOTE structures.
- Procedure
  - The challenger issues a challenge to the Application Enclave, which generates a manifest containing responses and an ephemeral public key.
  - After creating a REPORT structure, the QE signs it with an EPID private key.
  - The remote server validates the QUOTE signature and verifies manifest integrity.



Data Sealing Mechanisms

- Sealing to Enclave Identity
  - This policy produces a sealing key available exclusively to instances of a specific enclave, ensuring data can only be unsealed by the exact same enclave code.
- Sealing to Sealing Identity
  - This policy generates a key available to all enclaves signed by a particular sealing authority, enabling data sharing across related enclaves from the same developer.
- Both policies enable secure persistent storage of trusted data on external disks with confidentiality guarantees, allowing enclaves to securely store and retrieve sensitive information across execution sessions.

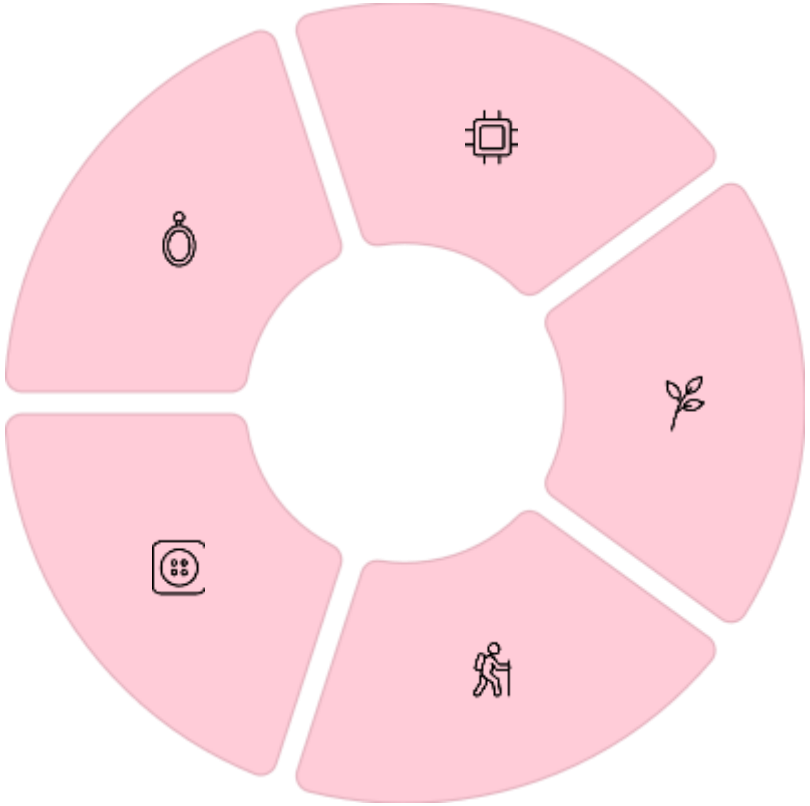
Attack Surface Categories

Page Table

Exploits side effects of page walking to observe memory access patterns at page granularity.

Enclave Interface

Analyzes ECALL/OCALL invocation patterns to infer sensitive input parameters.



CPU Cache

Leverages timing differences between cache hits and misses to infer execution patterns.

Branch Prediction

Abuses BTB, PHT, and RSB to extract control flow information at instruction level.

DRAM

Exploits row buffer timing to observe memory access patterns across virtual machines.

CPU Cache Vulnerabilities

- CPU caches bridge the speed gap between processors and DRAM, storing recently accessed data for fast retrieval. However, shared caches create timing channels that leak sensitive information about enclave execution.
- Cache controllers check whether requested data is cached (cache hit) or must be fetched from DRAM (cache miss). The measurable time difference between hits and misses enables attackers to infer which data the victim accessed, revealing secrets through careful timing analysis.

Prime+Probe Attack

- How It Works
  - **Initialization**: Attacker fills targeted cache sets with their own data, "priming" the cache.
  - **Waiting**: Victim process executes, potentially evicting attacker's data from cache sets.
  - **Measurement**: Attacker reloads their data and measures access time. Slower access indicates victim used that cache set.
- Requirements
  - Knowledge of LLC slice mapping
  - Ability to create eviction sets
  - No shared memory needed
  - Works in cross-core scenarios

Flush+Reload Attack

Flush+Reload is one of the most powerful cache attack techniques, requiring shared memory between attacker and victim but providing high precision and low noise.

Flush Phase

Attacker uses the clflush instruction to flush shared critical data out of all cache levels.

Victim Execution

Victim process executes its operations, potentially accessing the flushed data and bringing it back into cache.

Reload and Measure

Attacker reloads the shared data. Lower access time means victim accessed it; higher time means victim didn't.

Additional Attack Techniques

Evict+Time

Attacker measures victim's total execution time after evicting specific cache lines. Higher execution time indicates evicted lines were accessed.

Flush+Flush

Measures the execution time of the clflush instruction itself. Fast execution means data was already flushed; slow means it was in cache.

Evict+Reload

Combines eviction with reload measurement. Attacker evicts data, victim executes, then attacker checks if their data was replaced.

Same-Core vs. Cross-Core Attacks

Same-Core Attacks

Victim and attacker processes run on the same processing core, accessing the shared L1 cache. Since L1 caches are virtually indexed, attackers don't need to understand virtual-to-physical address mapping.

These attacks are simpler but require the attacker to be scheduled on the same core as the victim.

Cross-Core Attacks

Victim and attacker run on different cores, targeting the shared Last Level Cache (LLC). These attacks require knowledge of virtual-to-physical address mapping and LLC slice mapping functions.

Cross-core attacks are more challenging but more realistic in multi-tenant cloud environments.

Attacker Knowledge

Trace-Driven Attacks

Attacker knows which cache lines the victim accessed and the exact order of these accesses. This provides the most detailed information but is hardest to obtain.

Access-Driven Attacks

Attacker has snapshots of cache state at various instances. Can infer which cache lines were accessed but not their order. Most practical attack category.

Time-Driven Attacks

Attacker analyzes variations in execution time of security-sensitive applications. Different secrets cause different numbers of cache hits/misses, leading to timing variations.

Cache Coherence Protocol Attacks

- Coherence protocols maintain data consistency across cache levels in multi-core systems. Most commercial processors use MESI or MOESI protocols.
- These protocols create vulnerabilities because read-access latency differs for blocks in different coherent states.
- Attacker may change coherent state of blocks to alter access times and transfer information.

DRAM-Based Attacks

- DRAM attacks exploit timing differences in row buffer accesses without requiring memory sharing between attacker and victim. The row buffer holds the most recently accessed row, enabling faster access than other memory locations.
- While DRAM attacks achieve near Flush+Reload accuracy without shared memory, they face limitations: most memory resides in cache rather than DRAM, accuracy reaches only 1KB granularity (4 DRAM rows per 4KB page), and high noise levels from multiple pages sharing DRAM rows. DRAM attacks are typically combined with CPU cache attacks for improved effectiveness.

Address Translation Vulnerabilities

- The Translation Process
  - Intel's Memory Management Unit (MMU) translates virtual addresses to physical addresses through segmentation and paging units.
  - This essential mechanism creates attack surfaces that adversaries can exploit to observe memory access patterns.
- Vulnerability Sources
  - Page tables, segmentation units, and translation lookaside buffers (TLB) all maintain state information that can leak sensitive data about enclave execution through careful observation and manipulation.

Page-Fault Driven Attacks

Page-fault driven attacks exploit page fault exceptions to infer sensitive information. Attackers restrict page access by editing page tables, triggering faults when the victim accesses restricted pages. The chronological sequence of page faults reveals memory access patterns.

Locate Binary

Attacker infers the base address where target application binary is loaded in memory.

Restrict Access

Attacker modifies page tables to mark specific pages as inaccessible, setting the trap.

Monitor Faults

Each access to restricted pages triggers page faults, revealing which pages were accessed.

Extract Secrets

Analyzing fault patterns enables recovery of sensitive data like cryptographic keys.

While limited to page granularity (4KB), these attacks successfully extract large amounts of data including text documents, JPEG outlines, and cryptographic keys from implementations like OpenSSL and Libgcrypt.

Version Identification Attacks

Attack Principle

VID attacks identify target code running inside SGX enclaves by exploiting page access patterns, without requiring public access to victim code.

Information Revealed

Attackers can determine code algorithms, SDK library versions, and configuration information by observing execution patterns across numerous memory access events.

Security Impact

Experiments demonstrate observable differences in execution patterns between SDK versions and cryptographic algorithm modes, compromising code confidentiality.

Sneaky Page Monitoring

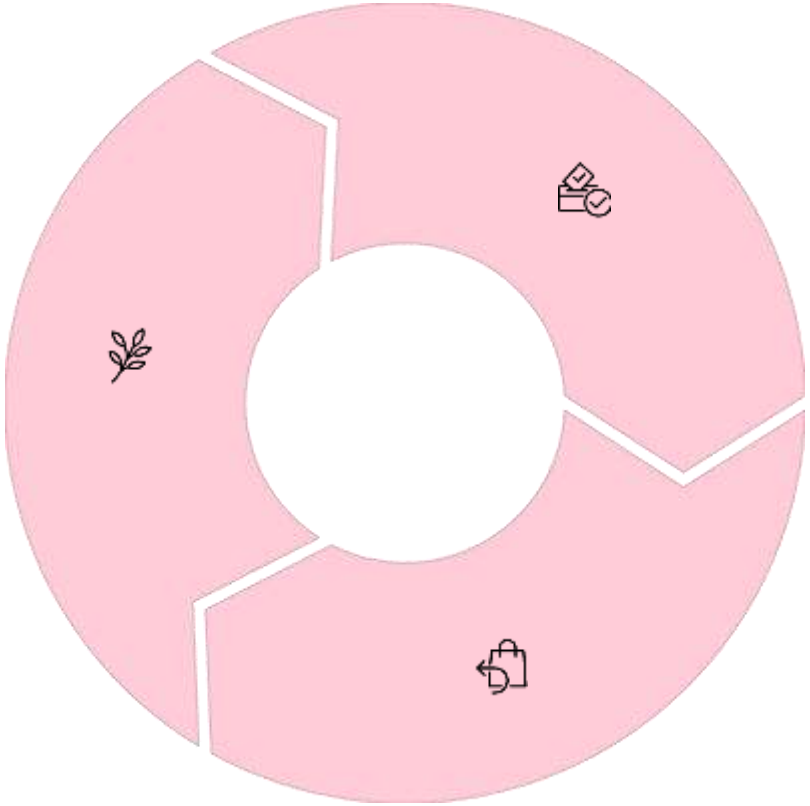
- SPM attacks achieve finer granularity than traditional page table attacks by combining multiple attack surfaces: page tables, CPU cache, and DRAM. This hybrid approach overcomes limitations of individual attack vectors.
- Key Innovation
  - SPM uses page accessed flags in PTEs to monitor page visits without direct interrupts. Combined with cache-DRAM techniques, it achieves cache-line level observation granularity.
- Efficiency Gains
  - By measuring execution time between entry and exit pages rather than tracking intermediate accesses, SPM reduces interrupts from 71,000 to just 1,300 for EdDSA attacks.

Branch Prediction

Modern processors use branch prediction to avoid pipeline stalls. The Branch Prediction Unit (BPU) predicts branch targets and outcomes before actual determination, enabling speculative execution that improves performance.

Branch Target Buffer

Stores computed target addresses of previously taken branch instructions for quick prediction.



Directional Predictor

Predicts whether branches will be taken or not taken based on historical patterns.

Return Stack Buffer

Predicts return addresses by maintaining a hardware stack of call instruction returns.

While improving performance, these prediction mechanisms create attack surfaces that leak sensitive control flow information.

PHT-Based Attacks

- Page History Table attacks exploit the directional branch predictor, creating collisions between victim and attacker process branches to infer branch directions and extract sensitive information.
- BranchScope
  - First attack exploiting PHT as an attack surface. Requires attacker and victim on same core, victim slowdown for high-resolution observation, and triggering many branches to activate vulnerable branches.
- Bluethunder
  - Evolutionary improvement creating collisions in 2-level predictors. Requires only single trigger of 2-level predictor versus many branches for bimodel predictor, achieving 52× faster attacks than BranchScope.

SgxPectre: Spectre Meets SGX

- SgxPectre extends the Spectre attack to compromise SGX enclaves by exploiting speculative execution channels. This attack manipulates the Branch Target Buffer to temporarily control enclave program control flow.
- The attack proceeds in five steps:
  - Poison the BTB to cause misprediction toward secret-leaking instructions
  - Increase probability of executing those instructions
  - Set registers for secret-leaking operations
  - Trigger enclave code execution
  - Extract secrets via cache channels using Flush+Reload
- Successfully demonstrated stealing seal keys and attestation keys from Intel-signed quoting enclaves.

RSB-Based Attacks

Return Stack Buffer attacks exploit the hardware stack used for predicting return addresses. Unlike BTB-based attacks, It pollutes the RSB to trigger mispredictions that leak sensitive information.

1

RSB Pollution

Attacker fills RSB with target addresses pointing to malicious payload gadgets containing secret-leaking instruction sequences.

2

Enclave Invocation

Attacker invokes specific enclave call to switch to trusted execution mode with polluted RSB state.

3

Unmatched Return

Enclave call contains unmatched return causing speculative execution at malicious target address.

4

Secret Extraction

Attacker exploits cache channel to infer leaked information from speculatively executed malicious gadgets.

Attacks succeed even against fully patched machines, demonstrating persistent security risks.

Enclave Interface Attacks

Enclave interface attacks exploit observable patterns in ECALL/OCALL function invocations. Attackers collect information about input parameter sizes and function invocation delays to infer sensitive data.

1 Pattern Collection

Monitor ECALL/OCALL invocations to gather timing and parameter size information.

2 Code Analysis

Analyze victim code in detail to understand interface usage patterns and data flows.

3 Core Isolation

Run victim code on isolated CPU core to improve measurement accuracy of invocation delays.

4 Secret Inference

Correlate observed patterns with code analysis to extract sensitive input parameters.

Demonstrated attacks against SGX-assisted network function virtualization platforms prove effectiveness of interface pattern analysis.